

Compte Rendu de TP

Introduction aux Scripts Bash

Administration Systeme - BTS SIO SISR

Auteur	Menad Yassine
Formation	BTS SIO - Option SISR
Module	Administration Systeme Linux
Date	Annee scolaire 2025-2026
Objectif	Creer un script Bash interactif servant de boite a outils pour l'administration Linux

Table des matieres

1. Introduction
2. Analyse et Recherche
 - 2.1 Le cours theorique du matin
 - 2.2 Decryptage du script d'exemple (learn.sh)
 - 2.3 Mes choix techniques
3. Planification de la journee
4. Le script final
 - 4.1 Architecture et structure
 - 4.2 Code complet annote
 - 4.3 Resultats et captures d'ecran
5. Bilan et competences acquises

1. Introduction

L'objectif de ce TP d'administration systeme (module SISR) etait de concevoir un script Bash fonctionnel servant de veritable boite a outils pour les commandes Linux courantes. Au-dela de la simple execution de commandes, l'enjeu etait de comprendre le principe fondamental de l'automatisation : comment enchainen des instructions de maniere structuree, interactive et robuste.

Un script Bash est, dans sa definition la plus simple, un fichier texte que l'interpreteur Shell lit ligne par ligne pour executer une serie de commandes. Pour un administrateur reseau ou systeme, c'est un outil indispensable : il permet d'automatiser les taches repetitives, de limiter les erreurs de frappe et de gagner un temps considerable au quotidien.

2. Analyse et Recherche

2.1 Le cours theorique du matin

La session du matin (11h - 12h) avec le formateur a pose les fondations theoriques necessaires. Trois concepts cles ont ete abordes :

- Le Shebang (`#!/bin/bash`) : premiere ligne obligatoire d'un script, elle indique au systeme quel interpreteur utiliser pour executer le fichier.
- La declaration de variables : comment stocker et reutiliser des valeurs au sein du script.
- Les structures de controle : les boucles et conditions qui permettent d'automatiser la logique de decision et de repetition.

2.2 Decryptage du script d'exemple (learn.sh)

En debut d'apres-midi, une analyse approfondie du script d'exemple fourni par le formateur a permis d'identifier trois mecanismes cles a reutiliser :

Mecanisme	Role et interet
Bloc H - <code>while true</code>	Menu persistant : maintient le script actif en continu apres chaque action, transformant un simple fichier de commandes en programme interactif.
Bloc D - <code>case</code>	Gestion des choix : structure la plus lisible pour traiter les saisies utilisateur (1, 2, 3... ou 'q'), bien plus propre qu'une serie de if/else imbriques.
Bloc I - <code>read -n1 -s</code>	Experience utilisateur : fonction de pause qui attend une touche sans afficher de saisie. Permet de lire les resultats avant que l'ecran ne se rafraichisse.

2.3 Mes choix techniques

Durant la phase de recherche, j'ai egalement explore la lecture de fichiers externes (fichiers .txt ou .csv avec `while IFS= read`). Apres avoir bien compris la logique, j'ai fait le choix delibere de ne pas l'integrer au script final.

Ce choix repose sur une logique professionnelle : mieux vaut livrer un outil plus simple mais 100% fonctionnel et robuste, plutot qu'un script plus ambitieux contenant des bugs ou des fonctionnalites incompletes. En contexte d'administration systeme, la fiabilite prime sur la complexite.

3. Planification de la journee

La journee a ete organisee de maniere structuree afin de respecter les contraintes de temps :

Horaire	Activite
11h00 - 12h00	Cours d'initiation aux scripts Bash avec le formateur : shebang, variables, boucles while et structures de conditions.
13h30 - 14h00	Analyse du script d'exemple learn.sh : identification des mecanismes cles (menu persistant, structure case, fonction de pause).
14h00 - 14h30	Developpement du script de boite a outils. Priorite donnee a la qualite et la robustesse du code sur la quantite de fonctionnalites.
14h30 - 14h50	Phase de validation : tests de fonctionnement, captures d'ecran, redaction du compte rendu.
15h00	Depot du travail finalise sur la plateforme.

4. Le script final

4.1 Architecture et structure

Le script est organise autour de trois composantes principales, directement inspirees de l'analyse du script d'exemple :

- Une fonction pause() : attend une pression de touche (silencieuse) avant de rafraichir l'ecran.
- Une boucle while true : maintient le menu actif en permanence jusqu'a ce que l'utilisateur choisisse de quitter.
- Une structure case : route chaque saisie vers la fonctionnalite correspondante de maniere claire et maintenable.

Le script propose 5 modules fonctionnels :

- Module 1 - Informations systeme : hostname, version OS (uname -a), utilisation memoire (free -h), espace disque (df -h), uptime.
- Module 2 - Gestion des utilisateurs : utilisateur courant (whoami), groupes (groups), liste des comptes systeme (/etc/passwd).
- Module 3 - Gestion des processus : top 10 CPU (ps aux), nombre total de processus.
- Module 4 - Informations reseau : interfaces IP (ip addr), routage (ip route), connexions actives (ss -tuln).
- Module 5 - Gestion des fichiers : repertoire courant (pwd), listing detaille (ls -lh), espace utilise (du -sh).

4.2 Code complet annotate

Voici le script Bash complet produit a l'issue du TP :

```
#!/bin/bash

# =====
```

```
# Boite a outils Linux - Administration Systeme
# Auteur : Menad Yassine | BTS SIO SISR
# =====

pause() {
    echo ""
    echo "Appuyez sur une touche pour continuer..."
    read -r -n 1 -s
}

while true; do
    clear
    echo "======"
    echo "  BOITE A OUTILS - Administration Linux"
    echo "======"
    echo "  1) Informations systeme"
    echo "  2) Gestion des utilisateurs"
    echo "  3) Gestion des processus"
    echo "  4) Informations reseau"
    echo "  5) Gestion des fichiers et repertoires"
    echo "  q) Quitter"
    read -r choix

    case $choix in
        1) clear; hostname; uname -a; free -h; df -h; uptime; pause ;;
        2) clear; whoami; groups; cut -d: -f1 /etc/passwd; pause ;;
        3) clear; ps aux --sort=-%cpu | head -11; pause ;;
        4) clear; ip addr show; ip route; ss -tuln; pause ;;
        5) clear; pwd; ls -lh; du -sh .; pause ;;
        q|Q) clear; echo 'Au revoir !'; exit 0 ;;
        *) echo "Choix invalide."; pause ;;
    esac
done
```

4.3 Description des commandes utilisees

Commande	Description
#!/bin/bash	Shebang : definit l'interpreteur a utiliser pour executer le script.
while true; do	Boucle infinie : maintient le menu actif jusqu'a la saisie de 'q'.
read -r choix	Lit la saisie de l'utilisateur et la stocke dans la variable \$choix.
case \$choix in	Structure de choix multiple, plus lisible que des if/else imbriques.
read -r -n 1 -s	Lit 1 caractere silencieusement (sans l'afficher) - utilisee dans pause().
clear	Efface le terminal pour un affichage propre avant chaque menu.
hostname	Affiche le nom de la machine.
uname -a	Affiche les informations completes du noyau et du systeme.
free -h	Affiche l'utilisation de la memoire RAM en format lisible.
df -h	Affiche l'espace disque disponible par partition.
uptime	Affiche la duree de fonctionnement du systeme.
whoami	Affiche le nom de l'utilisateur courant.
groups	Affiche les groupes auxquels appartient l'utilisateur courant.

<code>ps aux --sort=-%cpu</code>	Liste les processus triés par consommation CPU décroissante.
<code>ip addr show</code>	Affiche les interfaces réseau et leurs adresses IP.
<code>ip route</code>	Affiche la table de routage IP.
<code>ss -tuln</code>	Affiche les connexions réseau actives (TCP/UDP).
<code>ls -lh</code>	Liste le contenu du répertoire courant avec détails et taille lisible.
<code>du -sh .</code>	Affiche l'espace total occupé par le répertoire courant.

5. Bilan et compétences acquises

5.1 Ce que j'ai appris

Ce TP m'a permis de comprendre concrètement ce qu'est un script : derrière le jargon technique, c'est un fichier texte lu ligne par ligne par l'interpréteur Shell. La vraie valeur n'est pas dans la complexité du code, mais dans la logique d'organisation qu'il incarne.

En me mettant dans la peau d'un administrateur système, j'ai compris que le scripting est l'outil incontournable du quotidien : automatiser les vérifications récurrentes, centraliser les commandes utiles et réduire les risques d'erreur de saisie.

5.2 Compétences mises en oeuvre

- Écriture d'un script Bash structuré avec shebang, variables et fonctions.
- Utilisation de la boucle `while true` pour créer un menu interactif persistant.
- Utilisation de la structure `case` pour une gestion propre des choix utilisateur.
- Implémentation d'une fonction de pause avec `read -r -n 1 -s`.
- Utilisation des commandes d'administration : `ps`, `free`, `df`, `ip`, `ss`, `du`, `ls`.
- Application d'une démarche méthodique : analyse, planification, développement, validation.

5.3 Axes d'amélioration

Pour aller plus loin, ce script pourrait être enrichi avec les fonctionnalités suivantes :

- Lecture de fichiers de configuration externes (`.csv` ou `.conf`) avec `while IFS= read`.
- Ajout d'un module de gestion des services `systemd` (`systemctl status/start/stop`).
- Intégration de la journalisation : enregistrement des actions dans un fichier de log horodaté.
- Ajout de couleurs ANSI pour améliorer la lisibilité de l'interface.